Silicon Knights · Meeting 2

Combinational vs. Sequential Logic

Agenda

1. The big picture: memory or no memory
2. Combinational building blocks
3. Combinational pitfalls: loops and latches
4. Sequential building blocks
5. Clocks, resets and enables
6. Summary and questions

Objectives

Tell the two kinds of logic apart at a glance

Recognize the standard blocks in HDL

Avoid the mistakes Vivado warns about

Two Kinds of Digital Logic

COMBINATIONAL

```
out = f(inputs)
```



- No memory
- Same inputs → same outputs, always
- Gates only, no clock needed

SEQUENTIAL

```
out = f(inputs, state)
```



- Has memory (state)
- Output depends on history too
- Updates on a clock edge

PART 1

Combinational Logic

Circuits with no memory

Output Depends Only on Current Inputs

- Change an input → output changes after a tiny gate delay
- No clock, no stored value
- Any truth table can be built this way
- In SystemVerilog: **assign** or **always_comb** with blocking **=**

```
// continuous assignment
assign y = (a & b) | c;

// same logic, procedural style
always_comb begin
  y = (a & b) | c;
end
```

Multiplexers

A digital switch: **select** bits choose which input reaches the output.

sel	y
00	a
01	b
10	c
11	d

n select bits pick from 2^n inputs

```
// 2:1 mux
assign y = sel ? b : a;

// 4:1 mux
always_comb begin
  case (sel)
    2'b00: y = a;
    2'b01: y = b;
    2'b10: y = c;
    default: y = d;
  endcase
end
```

Decoders

Turns an n-bit code into 2^n lines, with exactly **one** line high (one-hot).

in	out
00	0001
01	0010
10	0100
11	1000

Uses: address decoding, picking one display digit, 7-segment drivers

```
// 2-to-4 decoder
always_comb begin
  out = 4'b0000;
  out[in] = 1'b1;
end

// or spelled out
assign out = (in == 2'd0) ? 4'b0001 :
             (in == 2'd1) ? 4'b0010 :
             (in == 2'd2) ? 4'b0100 :
                            4'b1000;
```

Adders

Half adder (2 bits in)

```
sum = a ^ b
carry = a & b
```

Full adder (adds carry-in)

```
sum = a ^ b ^ cin
cout = (a&b) | (cin&(a^b))
```

Chain n full adders = **ripple-carry** adder: each carry feeds the next bit

```
// 4-bit adder in one line
logic [3:0] a, b, sum;
logic cin, cout;

assign {cout, sum} = a + b + cin;

// {cout, sum} is 5 bits wide,
// so the carry-out is kept
```

Comparators

Compare two numbers and output single-bit flags.

- **Equality:** XNOR each bit pair, then AND them all
- **Less / greater:** a subtraction and check the sign or borrow

> **Watch out: signed vs. unsigned**
> 4'b1111 is 15 unsigned but −1 signed. Declare signals **signed** when you mean it.

```
logic [7:0] a, b;

assign eq = (a == b);
assign lt = (a <  b);
assign gt = (a >  b);

// equality by hand:
assign eq2 = &(~(a ^ b));
```

Priority Encoders

Several requests at once? Output the index of the **highest-priority** one.

req[3:0]	idx	valid
1xxx	3	1
01xx	2	1
001x	1	1
0001	0	1
0000	0	0

x = don't care. Uses: interrupts, bus arbiters

```
always_comb begin
  valid = |req;
  if      (req[3]) idx = 2'd3;
  else if (req[2]) idx = 2'd2;
  else if (req[1]) idx = 2'd1;
  else             idx = 2'd0;
end

// if/else order = priority
```

ALUs (Arithmetic Logic Units)

The heart of a CPU, built from the blocks we just saw:

- **Adders** for add and subtract
- **Gates** for AND, OR, XOR
- **Comparators** for less-than and zero
- **A mux** driven by the opcode

```
always_comb begin
  case (op)
    3'b000: result = a + b;
    3'b001: result = a - b;
    3'b010: result = a & b;
    3'b011: result = a | b;
    3'b100: result = a ^ b;
    3'b101: result = (a < b);
    default: result = 0;
  endcase
  zero = (result == 0);
end
```

Combinational Feedback Problems

A **combinational loop**: an output feeds back into its own input with no flip-flop in between.

- Can oscillate or settle unpredictably
- Timing tools can't analyze it
- Vivado flags it in DRC (combinatorial loop)

Fix

Break the loop with a register. Feedback belongs in sequential logic.

```
// BAD: x depends on itself
assign x = ~x;

// BAD: hidden loop a → b → a
assign a = b & c;
assign b = a | d;

// GOOD: loop goes through a FF
always_ff @(posedge clk)
```

Complete Signal Assignment

In an **always_comb** block, **every output** must get a value on **every path** through the code.

INCOMPLETE

```
always_comb begin
  if (en)
    y = a;
  // en = 0: y = ???
end
```

COMPLETE

```
always_comb begin
  if (en)
    y = a;
  else
    y = 1'b0;
end
```

Avoiding Unintended Latches

Three habits that prevent them:

1. **Default values** at the top of the block
2. An **else** for every **if**
3. A **default** in every **case**

Vivado warns: **[Synth 8-327] inferring latch**. Never ignore it.

always_comb states your intent, so tools flag a latch instead of silently building one.

```
always_comb begin
  // 1. defaults first
  y = 1'b0;
  z = 1'b0;

  case (sel)
    2'b00: y = a;
    2'b01: z = b;
    default: ; // 3.
  endcase
end

// y and z always have a value
```

PART 2

Sequential Logic

Circuits that remember

State

State = what the circuit remembers about the past. Same inputs, different history → different output.

Traffic light
Remembers which color it is showing now

Vending machine
Remembers how much money is inserted

Counter
Remembers the current count

```
next_state = f(state, inputs)  ← combinational logic, stored in flip-flops on each clock edge
```

Flip-Flops

The **D flip-flop** stores 1 bit. On the rising clock edge, Q copies D. Otherwise Q holds.

clk	Q next
rising edge ↑	D
anything else	Q (hold)

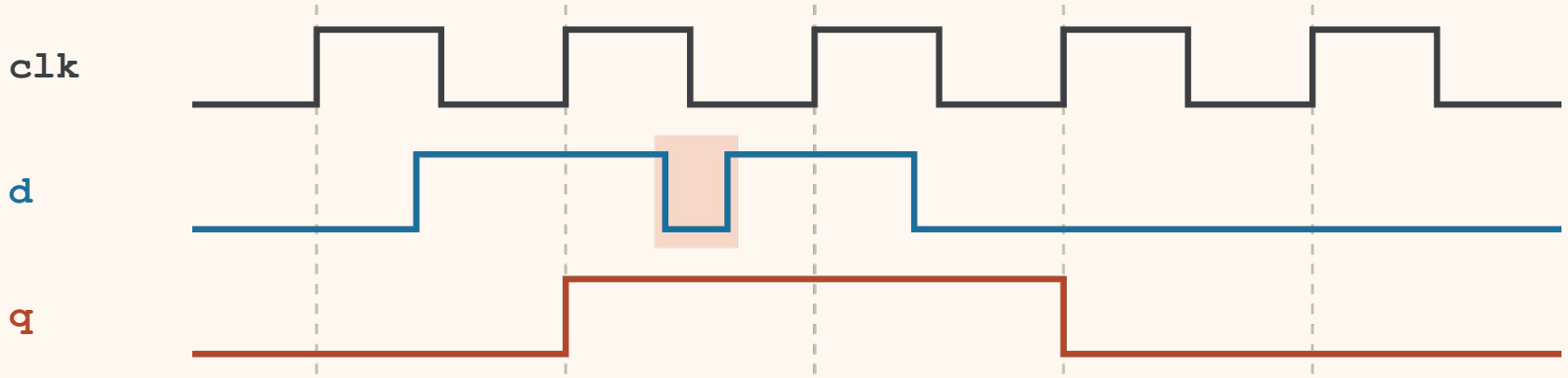
Latch = transparent while enable is high. **Flip-flop** = samples only at the edge. On FPGAs, use flip-flops.

```
// D flip-flop
logic q;

always_ff @(posedge clk)
  q <= d;

// posedge clk = rising edge
// <= is nonblocking (clocked)
```

Clocked State Updates



q changes **only at rising edges** (dashed lines), all flip-flops at the same instant

The short **glitch** on d (shaded) falls between edges, so q never sees it

Clocked Updates in Code: <= vs =

Nonblocking **<=** reads every old value first, then updates all at once, just like real flip-flops.

NONBLOCKING: SWAPS

```
always_ff @(posedge clk) begin
  a <= b;
  b <= a;
end
// a=1,b=2 → a=2,b=1
```

BLOCKING: DOESN'T

```
always_ff @(posedge clk) begin
  a = b;
  b = a;
end
// a=1,b=2 → a=2,b=2
```

Registers

A **register** is N flip-flops sharing one clock, storing an N-bit value.



8 flip-flops = one 8-bit register, all updated on the same edge

- Hold CPU values, pipeline stages, settings
- Vivado maps each bit to an FPGA flip-flop

```
// 8-bit register
logic [7:0] r;

always_ff @(posedge clk)
  r <= d;

// same code as one flip-flop,
// just a wider signal
```

Enable Signals

An **enable** decides whether the register loads a new value on this edge, or holds.

- Holding in a clocked block is fine: no latch, the flip-flop already remembers
- Slow things down with a 1-cycle enable pulse, not a new clock

Don't gate the clock

clk & en creates glitches and timing problems. Keep the board clock; use enables.

```
// GOOD: enable
always_ff @(posedge clk)
  if (en)
    q <= d;
  // no else: q holds

// BAD: gated clock
logic gclk;
assign gclk = clk & en;
always_ff @(posedge gclk)
  q <= d;
```

Synchronous Reset

Reset puts state into a known starting value.

Synchronous reset only acts **on a clock edge**.

- rst is just another input checked at the edge
- Clean timing: everything happens at the edge
- Generally recommended on AMD FPGAs
- Maps to FDRE flip-flops

```
always_ff @(posedge clk) begin
  if (rst)
    q <= 8'd0;
  else
    q <= d;
end

// only clk in the sensitivity
// list: reset waits for an edge
```

Asynchronous Reset

Asynchronous reset acts **immediately**, without waiting for the clock.

	Sync	Async
Acts	at clock edge	right away
Sensitivity	posedge clk	+ posedge rst
Needs clock	yes	no
FPGA cell	FDRE	FDCE

Releasing an async reset near a clock edge can cause trouble: sync the release.

```
always_ff @(posedge clk or
            posedge rst) begin
  if (rst)
    q <= 8'd0;
  else
    q <= d;
end

// rst in the sensitivity list
// = asynchronous
```

Counters

A register plus an adder feeding back into itself:

count ← count + 1 every edge.



- Timers and delays
- Clock-enable ticks: count to 99,999,999 at 100 MHz = 1 tick per second
- Addresses, scoreboards, PWM

```
logic [3:0] count;

always_ff @(posedge clk) begin
  if (rst)
    count <= 4'd0;
  else if (en)
    count <= count + 1;
end

// reset + enable + adder
// = everything so far
```

Shift Registers

Flip-flops in a chain: each edge, every bit moves one place and a new bit enters.

edge	din	sr[3:0]
start	–	0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011

Uses: serial ↔ parallel (UART, SPI), delay lines, LED chasers

```
logic [3:0] sr;

always_ff @(posedge clk)
  sr <= {sr[2:0], din};

// drop the top bit,
// shift left, din enters
// at bit 0
```

Side by Side

	Combinational	Sequential
Output depends on	current inputs	inputs + stored state
Memory	none	flip-flops
SV block	assign / always_comb	always_ff @(posedge clk)
Assignment	blocking =	nonblocking <=
Building blocks	mux, decoder, adder, ALU, comparator, priority encoder	register, counter, shift register
Classic bug	unintended latch, loops	gated clocks, wrong clock

Real designs use both: combinational logic computes, flip-flops remember.

Questions?

Next time: finite state machines